

SESSION 11

Working With Media Files & User Profile

Topics Covered: MEDIA_ROOT and MEDIA_URL, Upload profile picture, Serving uploaded files

Activities: Profile edit page (update photo, name, phone)

Task 1: Set up MEDIA_ROOT and MEDIA_URL in settings.py so uploaded media files stored in user_uploads folder accessed via /media/.

Explanation & Approach: *MEDIA_ROOT* is the filesystem path where uploads are saved; *MEDIA_URL* is the public browser URL prefix.

```
# settings.py
import os
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

MEDIA_URL = '/media/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'user_uploads')

# urls.py
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    # ... existing routes
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Task 2: Create Django model InfluencerProfile with fields: user (OneToOneField), display_name, bio, profile_pic (ImageField). Upload to profile_pics/.

Explanation & Approach: *ImageField* manages image uploads, validating file binary formats.

```
# models.py
from django.db import models
from django.contrib.auth.models import User

class InfluencerProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE, related_name='profile')
    display_name = models.CharField(max_length=100)
    bio = models.TextField(blank=True)
    profile_pic = models.ImageField(upload_to='profile_pics/', blank=True, null=True)
    phone_number = models.CharField(max_length=10, blank=True, null=True)

    def __str__(self):
        return f"{self.display_name} (@{self.user.username})"
```

Task 3: Build profile edit page where users update display name, bio, upload new profile picture (Instagram style).

Explanation & Approach: *Handling request.FILES along with request.POST is essential when forms include file/image uploads.*

```
# views.py
from django.shortcuts import render, redirect
from django.contrib.auth.decorators import login_required
from .models import InfluencerProfile

@login_required
def edit_profile(request):
    profile, created = InfluencerProfile.objects.get_or_create(user=request.user)

    if request.method == 'POST':
        profile.display_name = request.POST.get('display_name')
        profile.bio = request.POST.get('bio')

        if 'profile_pic' in request.FILES:
            profile.profile_pic = request.FILES['profile_pic']

        profile.save()
        return redirect('view_profile')

    return render(request, 'profile/edit_profile.html', {'profile': profile})
```

Task 4: Display uploaded profile picture on user profile page, falling back to default image if no picture uploaded.

Explanation & Approach: *Using {% if profile.profile_pic %} prevents template crashes when image fields are null.*

```
<!-- templates/profile/view_profile.html -->
<div class="card text-center p-4">
    {% if profile.profile_pic %}
        
    {% else %}
        
    {% endif %}
    <h3 class="mt-3">{{ profile.display_name }}</h3>
    <p class="text-muted">{{ profile.bio }}</p>
</div>
```

Task 5: Add phone number field to InfluencerProfile, update edit form to allow changing, validate phone number is exactly 10 digits.

Explanation & Approach: *Validates exact 10-digit numeric phone numbers before database commit.*

```
# forms.py
from django import forms
```

```
from django.core.exceptions import ValidationError
import re

class ProfileForm(forms.ModelForm):
    class Meta:
        model = InfluencerProfile
        fields = ['display_name', 'bio', 'profile_pic', 'phone_number']

    def clean_phone_number(self):
        phone = self.cleaned_data.get('phone_number')
        if phone:
            if not re.match(r'^\d{10}$', phone):
                raise ValidationError("Phone number must be exactly 10 numeric digits.")
        return phone
```